

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: M.Naga Doleswari Aruna Devi

Roll No.: A24126552267

Date: 29-08-2026

1. TITLE

Design an Interactive To-Do List Application using HTML, CSS and JavaScript

2. AIM

To design and develop an interactive To-Do List web application that allows a user to add, mark as complete, and delete tasks using HTML, CSS and JavaScript.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how HTML, CSS and JavaScript work together to build an interactive web page.
 - To style a webpage using internal CSS, including layout, colors, shadows and rounded corners.
 - To dynamically create and manipulate DOM elements using JavaScript.
 - To handle user events such as button clicks using event listeners.
 - To implement functionality for adding, completing and deleting tasks.
 - To conditionally show or hide content based on the state of the application.
-

4. THEORY

HTML provides the basic structure of a webpage, while CSS is used to style and visually enhance its appearance. JavaScript adds interactivity by allowing the webpage to respond to user actions such as clicks and text input, without requiring the page to reload.

The Document Object Model (DOM) represents the HTML page as a tree of objects that JavaScript can access and modify. Methods such as `document.createElement()` and `appendChild()` allow new elements to be created and inserted into the page dynamically, while `addEventListener()` allows JavaScript code to run in response to events such as a button click.

In this experiment, a To-Do List application is built where a user can type a task and click a button to add it to a list. Each task is displayed with a "Complete" button, which toggles a strikethrough style, and a "Delete" button, which removes the task from the list. A message is shown when the list is empty and hidden automatically once a task is added.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file and declare the HTML5 document type using `<!DOCTYPE html>`.
 2. Create the `<html>`, `<head>` and `<body>` sections, and add a title inside the `<title>` element.
 3. Define internal CSS inside a `<style>` tag to style the container, input box, buttons and task list.
 4. Create a container div with a heading, a text input field and an "Add Task" button.
 5. Add a paragraph to display a message when there are no tasks, and an empty unordered list to hold the tasks.
 6. Write a JavaScript block and select the input field, add button, task list and empty message using `getElementById`.
 7. Attach a click event listener to the "Add Task" button.
 8. Inside the event listener, read and trim the input value, and validate that it is not empty.
 9. Dynamically create a list item, a span for the task text, a "Complete" button and a "Delete" button using `createElement()`.
 10. Attach an event listener to the "Complete" button to toggle a CSS class that applies a strikethrough style.
 11. Attach an event listener to the "Delete" button to remove the task item from the list.
 12. Append the span and buttons to the list item, and append the list item to the task list.
 13. Clear the input field and update the visibility of the empty message using a helper function.
 14. Save the HTML file and open it in a web browser.
 15. Verify that tasks can be added, marked complete and deleted correctly.
 16. Compare the actual output with the expected output.
-

6. PROGRAM

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My To-Do List</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      text-align: center;
      padding: 40px;
    }

    .container {
      width: 500px;
      margin: auto;
      background-color: white;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0px 4px 10px gray;
    }

    input {
      width: 60%;
      padding: 10px;
      border: 1px solid gray;
      border-radius: 5px;
    }

    #addButton {
      padding: 10px 15px;
      background-color: green;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
    }

    ul {
      list-style-type: none;
      padding: 0;
    }

    li {
      display: flex;
      justify-content: space-between;
      align-items: center;
      background-color: #eeeeee;
      margin: 10px 0;
      padding: 10px;
      border-radius: 5px;
    }

    .complete {
      background-color: green;
      color: white;
      border: none;
      padding: 7px 10px;
      border-radius: 5px;
      cursor: pointer;
    }

    .delete {
      background-color: red;
      color: white;
      border: none;
      padding: 7px 10px;
      border-radius: 5px;
      cursor: pointer;
      margin-left: 5px;
    }

    .completed {
      text-decoration: line-through;
      color: gray;
      opacity: 0.6;
    }

    #emptyMessage {
      color: gray;
      margin-top: 20px;
    }
  </style>
</head>

<body>

  <div class="container">

    <h1>My To-Do List</h1>

    <input
      type="text"
      id="taskInput"
      placeholder="Enter a task..."
    >

```

```

<button id="addButton">Add Task</button>

<p id="emptyMessage">No tasks available.</p>

<ul id="taskList"></ul>

</div>

<script>

// Select DOM elements
const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Add Task Event
addButton.addEventListener("click", function () {

    // Get task value
    const taskText = taskInput.value.trim();

    // Check if input is empty
    if (taskText === "") {
        alert("Please enter a task!");
        return;
    }

    // Create new list item
    const taskItem = document.createElement("li");

    // Create task text
    const taskName = document.createElement("span");

    taskName.textContent = taskText;

    // Create Complete Button
    const completeButton = document.createElement("button");

    completeButton.textContent = "Complete";

    completeButton.className = "complete";

    // Complete button event
    completeButton.addEventListener("click", function () {

        taskName.classList.toggle("completed");

    });

    // Create Delete Button
    const deleteButton = document.createElement("button");

    deleteButton.textContent = "Delete";

    deleteButton.className = "delete";

    // Delete button event
    deleteButton.addEventListener("click", function () {

        taskItem.remove();

        checkEmptyList();

    });

    // Add elements to list item
    taskItem.appendChild(taskName);

    taskItem.appendChild(completeButton);

    taskItem.appendChild(deleteButton);

    // Add task to webpage
    taskList.appendChild(taskItem);

    // Clear input box
    taskInput.value = "";

    // Hide empty message
    checkEmptyList();

});

// Function to check if task list is empty
function checkEmptyList() {

```

```

        if (taskList.children.length === 0) {

            emptyMessage.style.display = "block";

        } else {

            emptyMessage.style.display = "none";

        }

    }

}

</script>

</body>
</html>

```

7. CODE EXPLANATION

Keyword / Element	Description
<style>	Defines internal CSS used to style the container, input box, buttons and list items.
.container	CSS class that styles the white card holding the to-do list, including padding, border radius and shadow.
box-shadow	CSS property that adds a soft shadow around the container to create a raised card effect.
.complete / .delete	CSS classes that style the Complete (green) and Delete (red) buttons attached to each task.
.completed	CSS class that applies a strikethrough and reduced opacity to mark a task as done.
getElementById()	JavaScript method used to select the input field, add button, task list and empty message elements.
addEventListener()	Attaches a function to an element that runs when a specific event, such as a click, occurs.
trim()	Removes leading and trailing whitespace from the entered task text before validation.
createElement()	Dynamically creates new HTML elements such as the list item, span and buttons for each task.
classList.toggle()	Adds or removes the "completed" class on a task to switch its strikethrough style on and off.
appendChild()	Inserts a newly created element into the DOM, such as adding a task item to the task list.
remove()	Removes a task's list item from the DOM when the Delete button is clicked.
checkEmptyList()	Custom function that shows or hides the "No tasks available" message based on the number of tasks.

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open the HTML file in a web browser with no tasks added	"No tasks available." message should be displayed	"No tasks available." message displayed	Pass
TC-02	Click "Add Task" without entering any text	An alert should prompt the user to enter a task	Alert "Please enter a task!" displayed	Pass
TC-03	Enter a task name and click "Add Task"	Task should appear in the list and empty message should disappear	Task added and empty message hidden	Pass
TC-04	Click "Complete" on a task	Task text should appear with a strikethrough	Task text struck through	Pass
TC-05	Click "Complete" again on the same task	Strikethrough should be removed	Strikethrough removed	Pass
TC-06	Click "Delete" on a task	Task should be removed from the list	Task removed from the list	Pass
TC-07	Delete all tasks from the list	"No tasks available." message should reappear	Empty message reappeared	Pass

9. OUTPUT

Empty to-do list on page load:

My To-Do List

Add Task

No tasks available.

To-do list after adding tasks:

My To-Do List

Add Task

record

CompleteDelete

observation

CompleteDelete

daily verse

CompleteDelete

prayer

CompleteDelete

10. RESULT

Thus, an interactive To-Do List web application was successfully designed and developed using HTML, CSS and JavaScript. The application was verified for adding tasks, marking tasks as complete, deleting tasks, and displaying the empty-list message correctly.